



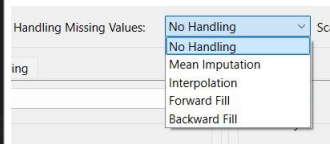
Machine Learning Gui

Mustafa Yarkin Konma

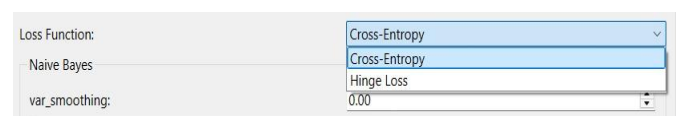
2206A605

I have added Missing Value Handling as shown in the picture with its code

```
def handle_missing_values(self):
    try:
        method = self.missing_values_combo.currentText()
        if method == "No Handling":
            return
        elif method == "Mean Imputation":
            imputer = SimpleImputer(strategy="mean")
            self.X_train = imputer.fit_transform(self.X_train)
            self.X_test = imputer.transform(self.X_test)
        elif method == "Interpolation":
            self.X_train = pd.DataFrame(self.X_train).interpolate().values
            self.X_test = pd.DataFrame(self.X_test).interpolate().values
        elif method == "Forward Fill":
            self.X_train = pd.DataFrame(self.X_train).fillna(method="ffill").values
            self.X_test = pd.DataFrame(self.X_test).fillna(method="ffill").values
        elif method == "Backward Fill":
            self.X_train = pd.DataFrame(self.X_train).fillna(method="bfill").values
            self.X_test = pd.DataFrame(self.X_test).fillna(method="bfill").values
    except Exception as e:
        print(f"error in handling missing values: {e}")
```



I have added Loss option to both classification and regression



GaussianNB

var_smoothing: 0.00

Prior Probabilities: User-Defined

Train GaussianNB

GaussianNB with var smoothing and prior probabilities

I have added Svm Modeling but for some reason it doesn't work

```
elif model_name == "Support Vector Machine":  
    C = param_widgets["C"].value()  
    kernel = param_widgets["kernel"].currentText()  
    degree = param_widgets["degree"].value() if kernel == "poly" else 2  
    model = SVC(kernel=kernel, C=C, degree=degree)  
    model.fit(self.X_train, self.y_train)  
    y_pred = model.predict(self.X_test)  
    self.current_model = model  
    loss_function = self.loss_classification_combo.currentText()  
    if loss_function == "Cross-Entropy":  
        loss = log_loss(self.y_test, model.decision_function(self.X_test))  
    elif loss_function == "Hinge loss":  
        loss = hinge_loss(self.y_test, y_pred)  
  
    self.update_visualization(y_pred)  
    self.update_metrics(y_pred, loss)
```

The code in picture